1) Andrew Branum: 5291638
2) I only implemented Parts 1-3. I attempted Part 4 but was unable to resolve it in a timely manner.
3) Questions:
    a. How are the execution times of the different parts compared to each other for the small input size?
        i. After running the tests multiple times, the UNIX and GO implementations had the fastest execution times for small files. This is expected since UNIX simply executes a command within the terminal.
    b. What happens when the input file size gets larger?
        i. As files get larger, UNIX and MR_S slow down while the GO implementation only increases ~3.85 seconds rather than 20+ seconds. The GO implementation wins this round, but I would assume that the DMP would be the fastest in this case if implemented.
    c. What is your vision for the execution time of different parts when the input size gets even larger, into gigabyte, terabyte, or even petabyte range?
        i. I think DMP would be the only viable option for files larger than gigabytes. It was created with these file sizes in mind so they can handle it accordingly.
    d. What is the scalability limitation of the SMP solution, and how DMP solution can eliminate this limitation?
        i. SMP is bounded to the resources on a single machine whereas DMP can use an infinite number of machines allowing for horizontal scaling rather than vertical seen in SMP.
    e. Which solution would you go with if the input file size was small?
        i. UNIX or GO as most execution times were near instant
    f. Which solution would you go with if the input file size was large?
        i. Depending on the computer and exact size of the file. If I have a beefy computer with small-large to medium-large I'd use SMP to keep complexity reduced. If the file was massive or I had a mediocre computer I'd opt for DMP
    g. Which solution would you go with if the input file size was small, but there were many input files to be processed?
        i. DMP or SMP depending on the number of files and the computer that is running the application specs. SMP if the computer can handle it but DMP otherwise.

4) Outputs:

```
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v
=== RUN   TestWordCount_UNIX_1
--- PASS: TestWordCount_UNIX_1 (0.03s)
=== RUN   TestWordCount_GO_1
--- PASS: TestWordCount_GO_1 (0.02s)
=== RUN   TestWordCount_MR_S_1_M4_R4
--- PASS: TestWordCount_MR_S_1_M4_R4 (0.07s)
=== RUN   TestWordCount_MR_SMP_1_M4_R4
    p3_test.go:50: WordCount_MR_SMP_1_M4_R4("THEGODFATHER.dat", "THEGODFATHER_MR_SMP_1_M4_R4_WC.out")
--- FAIL: TestWordCount_MR_SMP_1_M4_R4 (0.00s)
=== RUN   TestWordCount_MR_SMP_1_M8_R8
    p3_test.go:60: WordCount_MR_SMP_1_M8_R8("THEGODFATHER.dat", "THEGODFATHER_MR_SMP_1_M8_R8_WC.out")
--- FAIL: TestWordCount_MR_SMP_1_M8_R8 (0.00s)
=== RUN   TestWordCount_MR_DMP_1_M8_R8
    p3_test.go:70: WordCount_MR_DMP_1_M8_R8("THEGODFATHER.dat", "THEGODFATHER_MR_DMP_1_M8_R8_WC.out")
--- FAIL: TestWordCount_MR_DMP_1_M8_R8 (0.00s)
=== RUN   TestWordCount_MR_DMP_1_M16_R16
    p3_test.go:80: WordCount_MR_DMP_1_M16_R16("THEGODFATHER.dat", "THEGODFATHER_MR_DMP_1_M16_R16_WC.out")
--- FAIL: TestWordCount_MR_DMP_1_M16_R16 (0.00s)
=== RUN   TestWordCount_UNIX_1000
--- PASS: TestWordCount_UNIX_1000 (20.31s)
=== RUN   TestWordCount_GO_1000
--- PASS: TestWordCount_GO_1000 (3.89s)
=== RUN   TestWordCount_MR_S_1000_M4_R4
--- PASS: TestWordCount_MR_S_1000_M4_R4 (47.12s)
=== RUN   TestWordCount_MR_SMP_1000_M4_R4
    p3_test.go:120: WordCount_MR_SMP_1000_M4_R4("THEGODFATHER1000.dat", "THEGODFATHER_MR_SMP_1000_M4_R4_WC.out")
--- FAIL: TestWordCount_MR_SMP_1000_M4_R4 (0.00s)
=== RUN   TestWordCount_MR_SMP_1000_M8_R8
    p3_test.go:130: WordCount_MR_SMP_1000_M8_R8("THEGODFATHER1000.dat", "THEGODFATHER_MR_SMP_1000_M8_R8_WC.out")
--- FAIL: TestWordCount_MR_SMP_1000_M8_R8 (0.00s)
=== RUN   TestWordCount_MR_DMP_1000_M8_R8
    p3_test.go:140: WordCount_MR_DMP_1000_M8_R8("THEGODFATHER1000.dat", "THEGODFATHER_MR_DMP_1000_M8_R8_WC.out")
--- FAIL: TestWordCount_MR_DMP_1000_M8_R8 (0.00s)
=== RUN   TestWordCount_MR_DMP_1000_M16_R16
    p3_test.go:150: WordCount_MR_DMP_1000_M16_R16("THEGODFATHER1000.dat", "THEGODFATHER_MR_DMP_1000_M16_R16_WC.out")
--- FAIL: TestWordCount_MR_DMP_1000_M16_R16 (0.00s)
=== RUN   TestCleanFiles
--- PASS: TestCleanFiles (0.00s)
FAIL
exit status 1
FAIL    proj3   71.443s
```

Small files:

```
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run  ^TestWordCount_UNIX$
testing: warning: no tests to run
PASS
ok      proj3   0.002s
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_UNIX_1$
=== RUN   TestWordCount_UNIX_1
--- PASS: TestWordCount_UNIX_1 (0.03s)
PASS
ok      proj3   0.030s
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_GO_1$
=== RUN   TestWordCount_GO_1
--- PASS: TestWordCount_GO_1 (0.02s)
PASS
ok      proj3   0.020s
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_MR_S_1_M4_R4$
=== RUN   TestWordCount_MR_S_1_M4_R4
--- PASS: TestWordCount_MR_S_1_M4_R4 (0.11s)
PASS
ok      proj3   0.115s
```

Big Files:

```
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_UNIX_1000$
=== RUN   TestWordCount_UNIX_1000
--- PASS: TestWordCount_UNIX_1000 (20.37s)
PASS
ok      proj3   20.371s
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_GO_1000$
=== RUN   TestWordCount_GO_1000
--- PASS: TestWordCount_GO_1000 (3.85s)
PASS
ok      proj3   3.857s
ubuntu@ip-172-31-2-239:~/go/src/proj3$ go test -v -run ^TestWordCount_MR_S_1000_M4_R4$
=== RUN   TestWordCount_MR_S_1000_M4_R4
--- PASS: TestWordCount_MR_S_1000_M4_R4 (47.03s)
PASS
ok      proj3   47.037s
```